DBMS Chapter 20

Introduction to Transaction Processing Concepts and Theory

Comprehensive Exam Notes | Elmasri & Navathe, 7th Ed.

---

1. Transaction Basics

1.1 What is a Transaction?

A transaction is a logical unit of database processing — a set of one or more operations that must be executed as a single, indivisible unit.

Key Rule
A transaction must either FULLY SUCCEED (COMMIT) or FULLY FAIL (ROLLBACK). No partial execution is allowed.

Types of Transactions

- Read-Only Transaction
  - Only reads data from the database — does not modify it.
- Read-Write Transaction
  - Reads data and also writes (modifies) data in the database.

1.2 Single-User vs. Multiuser DBMS

Type	Description
Single-User DBMS	At most one user at a time can use the system. Example: personal home computer database.
Multiuser DBMS	Many users can access the database concurrently. Example: airline reservation systems, banking systems.

1.3 Multiprogramming & Concurrency

- Multiprogramming
  - OS executes multiple processes concurrently by switching between them.
- Interleaved Processing
  - A single CPU switches between multiple transactions, giving the illusion of concurrency.
- Parallel Processing
  - Multiple CPUs execute different transactions simultaneously at the same instant.

1.4 Database Items & Read/Write Operations

A database is represented as a collection of named data items. The size of a data item is called its granularity (could be a record, disk block, or attribute value).

Operation	Description
read_item(X)	Finds the disk block containing X, copies to memory buffer, assigns value to program variable X.
write_item(X)	Finds the disk block containing X, copies from memory buffer to disk, stores the updated block.
Read Set	The complete set of ALL data items read by a transaction.
Write Set	The complete set of ALL data items written by a transaction.

Example Transaction T1 (Bank Transfer)

```
read_item(X);   X := X - N;   write_item(X);   read_item(Y);   Y := Y + N;
write_item(Y);
```

T1 reads X, subtracts N (debit), writes X back, reads Y, adds N (credit), writes Y back.

2. ACID Properties of Transactions

Every valid transaction must satisfy the ACID properties to ensure database correctness and reliability.

ACID Property	Explanation
Atomicity (All or Nothing)	The entire transaction is treated as a single unit. Either ALL operations succeed and are committed, or NONE of them take effect (rollback). If a crash occurs midway, changes are undone.
Consistency Preservation	A transaction takes the database from one valid (consistent) state to another valid state. Integrity constraints must hold before and after the transaction.

Isolation	Concurrent transactions execute as if they were running serially. The intermediate state of a transaction is not visible to other transactions. Controlled by concurrency control mechanisms.
Durability (Permanency)	Once a transaction is committed, its changes are permanent even if a system crash occurs immediately after. Achieved via the system log and recovery mechanisms.

3. Transaction States & System Concepts

3.1 Transaction State Diagram

State	Description & Transitions
Active	Initial state when the transaction begins. Stays active while executing READ/WRITE operations.
Partially Committed	After the LAST operation executes (END_TRANSACTION). Changes are in memory but not yet permanently on disk.
Committed	Transaction successfully completes. All changes are permanently recorded in the database. CANNOT be rolled back.
Failed	Normal execution cannot proceed. Transaction has been aborted either from Active or Partially Committed state.
Terminated	Final state. Transaction either committed (success path) or aborted (failure path) and removed from system.

Transaction Operations
`BEGIN_TRANSACTION` → marks start
`READ / WRITE` → database operations during Active state
`END_TRANSACTION` → signals completion of all operations (moves to Partially Committed)
`COMMIT_TRANSACTION` → makes changes permanent (moves to Committed)
`ROLLBACK / ABORT` → undoes all changes (moves to Failed)

3.2 The System Log

The system log (journal) is a sequential, append-only file on disk that records ALL transaction operations. It is the foundation of crash recovery.

- Records: start_transaction, read, write, commit, abort for each transaction
- Not normally affected by transaction failures (only disk failure or catastrophe can destroy it)

- Log buffer: a main-memory buffer; written to disk when full or at commit point
- Backed up periodically to protect against disk failure
- Supports UNDO (roll back uncommitted changes) and REDO (replay committed changes)

Commit Point

A transaction reaches its commit point when:

1. All database operations have completed successfully.
2. The effect of all operations has been recorded in the system log.
3. A commit record is written to the log.

Force-writing: the log buffer must be flushed to disk BEFORE the transaction reaches commit point.

4. Concurrency Control Problems

When transactions execute concurrently without control, several critical problems can arise.

4.1 Lost Update Problem

Two transactions read and update the same item. One update overwrites the other, making it as if the first update never happened.

Step	T1 (Withdraw N)	T2 (Deposit M)
1	`read_item(X)`	
2	`X := X - N`	
3		`read_item(X)`
4		`X := X + M`
5	`write_item(X)`	
6		`write_item(X)` ← T1's update LOST!

Why it happens

T2 reads X before T1 writes back. T1 then writes its updated value, but T2 overwrites it with its own update (based on the old X). T1's change is permanently lost.

4.2 Dirty Read Problem (Temporary Update)

A transaction reads a value written by another transaction that later fails and rolls back. The reading transaction has used a value that never officially existed.

Step	T1	T2
1	read_item(X = 100)	
2	X := X - 50	
3	write_item(X = 50)	
4		read_item(X = 50) ← DIRTY READ
5		X := X + 10; write_item(X)
6	ROLLBACK (X → 100)	
7		Commit (used wrong X!)

Why it matters

T2 reads a value that was never officially committed. When T1 rolls back, T2 has already committed using a 'phantom' value. This leads to database inconsistency.

4.3 Incorrect Summary Problem

An aggregate transaction (e.g., computing SUM) reads some items before and some after another transaction updates them, producing a wrong result.

Step	T1 (Transfer N: X→Y)	T3 (Compute Sum)
1		sum := 0; read_item(A); sum += A
2	read_item(X); X:=X-N; write_item(X)	
3		read_item(X) ← reads AFTER debit
4		sum += X; read_item(Y) ← reads BEFORE credit
5		sum += Y; write_item(sum) WRONG by N!
6	read_item(Y); Y:=Y+N; write_item(Y)	

Result

T3 reads X after N is subtracted but reads Y before N is added. The sum is therefore off by N. The total money in the system appears to have decreased!

4.4 Unrepeatable Read Problem

A transaction reads the same item twice but gets different values because another transaction modified it between the two reads.

Step	T (reads X twice)	T' (modifies X)
1	`read_item(X) = 100`	
2		`X := X + 50;` `write_item(X)`
3		**`Commit`**
4	`read_item(X) = 150 ← DIFFERENT!`	

4.5 Phantom Read Problem

A transaction re-executes a query and finds new rows (phantoms) that appeared because another transaction inserted records between the two query executions.

Example
T1 queries: SELECT * FROM Employees WHERE dept='CS' → gets 5 rows.
T2 inserts a new CS employee and commits.
T1 re-queries: SELECT * WHERE dept='CS' → now gets 6 rows (a 'phantom').

5. Schedules (Histories)

5.1 What is a Schedule?

A schedule (or history) defines the ORDER in which operations from multiple concurrent transactions are executed. In a schedule, operations from different transactions can be interleaved, but the order of operations within a single transaction must be preserved.

Key Point
For any two operations in a schedule, one must occur before the other (total ordering). For n transactions, there are n! possible serial schedules.

5.2 Serial vs. Non-Serial Schedules

Type	Description
Serial Schedule	Transactions execute one after another with NO interleaving. Always produces a correct (consistent) result. But limits concurrency and performance. For n transactions: n! possible serial schedules.
Non-Serial Schedule	Operations from different transactions are interleaved. Improves performance and response time. May or may not produce a correct result (depends on whether it is serializable).

Example: Serial Schedule A (T1 then T2):

T1	T2
`read_item(X)`	
`X := X - N; write_item(X)`	
`read_item(Y); Y:=Y+N; write_item(Y)`	
	`read_item(X); X:=X+M; write_item(X)`

Example: Non-Serial Schedule C (interleaved — may or may not be correct):

T1	T2
`read_item(X)`	
`X := X - N`	
	`read_item(X); X:=X+M`
`write_item(X)`	
`read_item(Y); Y:=Y+N; write_item(Y)`	
	`write_item(X)`

5.3 Conflicting Operations

Two operations CONFLICT if ALL of the following are true:

- They belong to DIFFERENT transactions
- They access the SAME data item X
- At LEAST ONE of the operations is a write_item(X)

Conflict Type	Example
Read-Write Conflict (RW)	T1: read_item(X) — T2: write_item(X). T2's write affects what T1 reads (or vice versa depending on order).
Write-Read Conflict (WR)	T1: write_item(X) — T2: read_item(X). T2 reads the value written by T1 (dirty read risk).
Write-Write Conflict (WW)	T1: write_item(X) — T2: write_item(X). One write overwrites the other (lost update risk).

NOT a conflict: Read-Read	T1: read_item(X) — T2: read_item(X). Two reads never conflict — order doesn't matter.

6. Serializability

6.1 Conflict Serializability

A schedule S is conflict serializable if it is conflict-equivalent to some serial schedule S'. Two schedules are conflict-equivalent if the relative order of ALL conflicting operations is the same in both schedules.

Algorithm: Testing Conflict Serializability (Precedence Graph)

4. Create a node for each transaction Ti in the schedule.
5. Draw edge Ti → Tj if Tj reads after Ti writes the same item X (WR conflict).
6. Draw edge Ti → Tj if Tj writes after Ti reads the same item X (RW conflict).
7. Draw edge Ti → Tj if Tj writes after Ti writes the same item X (WW conflict).
8. **The schedule is conflict serializable ⇔The precedence graph has NO CYCLES.**

If the graph has no cycle, the serial order = topological sort of the graph.

Example: Conflict Serializability with 3 Transactions

T1	T2	T3
READ(X)		
	READ(Y)	
		READ(Y)
	WRITE(Y)	
WRITE(X)		
		WRITE(X)
	READ(X)	
	READ(X)	

Graph Analysis for above schedule

`Conflicts on X: T1 W(X) before T3 W(X) → T1→T3 | T3 W(X) before T2 R(X) → T3→T2`

Conflicts on Y: T2 W(Y) before T3 R(Y)? No—T3 reads Y before T2 writes Y → T3→T2 (T3 R(Y) then T2 W(Y))

Edges: T1→T3, T3→T2 | No cycles → Conflict Serializable!
Serial Order = T1 → T3 → T2

6.2 View Serializability

A schedule S is view serializable if it is view-equivalent to some serial schedule. Two schedules are view-equivalent if — for every data item X — the same transactions perform the initial read, the same read-from relationships hold, and the same transaction performs the final write.

Three Conditions for View Equivalence
9. Initial Read: If Ti reads the initial value of X in S, then Ti must also read the initial value of X in S'.
10. Read-From: If Ti reads a value written by Tj in S, then Ti must read the value written by Tj in S'. 11. Final Write: If Ti performs the final write on X in S, then Ti must also perform the final write on X in S'.
Note: Every conflict serializable schedule is also view serializable. But some view serializable schedules are NOT conflict serializable (those containing blind writes).

6.3 Result Equivalence (Not Sufficient Alone)

Two schedules are result equivalent if they produce the same final database state. However, this equivalence may be accidental (only for specific initial values) and cannot be used as the sole criterion for correctness.

7. Recoverability of Schedules

A schedule is classified based on how easily it can recover from transaction failures.

Schedule Type	Definition & Key Property
Recoverable Schedule	A transaction Ti that reads a value written by Tj may ONLY commit AFTER Tj commits. No committed transaction ever needs to be rolled back. This is the MINIMUM requirement — DBMSs must enforce this.
Cascadeless Schedule (Avoids Cascading Rollback)	A transaction Ti can only read values written by already-COMMITTED transactions. Eliminates the cascade effect where rolling back one transaction forces rollback of others.

Strict Schedule	Transactions can neither read NOR write an item X until the last transaction that wrote X has committed or aborted. Simplest to implement recovery (restore before-image). Strongest guarantee.

Hierarchy (from weakest to strongest)

Recoverable ⊆ Cascadeless ⊆ Strict ⊆ Serial

Every strict schedule is cascadeless. Every cascadeless schedule is recoverable. Every serial schedule is strict.

Non-Recoverable (IRRECOVERABLE) — Must NEVER be allowed:

Step	T1	T2
1	`R(X); X:=X+2; W(X)`	
2		`R(X); X:=X+3; W(X)`
3		Commit ← T2 commits BEFORE T1!
4	FAILED / ROLLBACK	
	IRRECOVERABLE: Cannot undo T2 since it already committed.	

Recoverable Schedule (T1 commits before T2):

Step	T1	T2
1	`R(X); X:=X+2; W(X)`	
2		`R(X); X:=X+3; W(X)`
3	**Commit**	
4		Commit ✓ T2 commits AFTER T1

8. Isolation Levels in SQL

SQL defines four isolation levels that control the trade-off between performance and correctness. Higher isolation = fewer anomalies but lower concurrency.

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes
REPEATABLE READ	No	No	Yes

SERIALIZABLE	No	No	No

Isolation Level	Description
READ UNCOMMITTED (Level 0)	Allows reading uncommitted data (dirty reads). Highest performance, lowest isolation. Rarely used.
READ COMMITTED (Level 1)	Only reads committed data. Prevents dirty reads but allows non-repeatable reads and phantoms.
REPEATABLE READ (Level 2)	All reads are repeatable. Prevents dirty reads and non-repeatable reads. Phantoms still possible.
SERIALIZABLE (Level 3)	Full isolation. Prevents all anomalies. Equivalent to serial execution. Highest isolation, lowest concurrency.

Snapshot Isolation
Used by many commercial DBMSs (Oracle, PostgreSQL). A transaction reads data based on the committed snapshot of the database at the time the transaction STARTED.
Guarantees: No dirty reads, no non-repeatable reads, NO phantom reads. Does NOT guarantee full serializability (write skew anomaly possible).

9. Why Recovery is Needed — Types of Failures

Failure Type	Description & Examples
Computer/System Crash	Hardware or OS failure while transactions are active. Main memory is lost; disk data may survive.
Transaction/System Error	Logic error, division by zero, deadlock detection forces abort. Transaction rolls back.
Local Errors / Exceptions	Application detects invalid state (e.g., insufficient funds, wrong date). Transaction aborts itself.
Concurrency Control Enforcement	DBMS aborts a transaction to prevent deadlock or serialization violation.
Disk Failure	Read/write error or disk crash. Requires restore from backup + log replay. Long recovery time.
Physical Catastrophes	Fire, flood, power outage destroying hardware. Requires off-site backup restoration.

10. Exam Questions with Solutions

Q1: Define a transaction. What are the four ACID properties? Explain each briefly.

Answer: Answer:

A transaction is a logical unit of database processing that must be executed in its entirety or not at all. It forms an indivisible unit of work.

ACID Properties:ACID Properties:

- Atomicity: Either ALL operations in the transaction execute successfully, or NONE do. If any operation fails, the entire transaction is rolled back.
- Consistency: A transaction transforms the database from one consistent (valid) state to another consistent state. All integrity constraints must hold before and after.
- Isolation: Concurrent transactions execute as if they are running serially. Intermediate states of a transaction are hidden from other concurrent transactions.
- Durability: Once committed, the effects of a transaction are permanent — they survive system crashes and failures. Ensured via logs and checkpoints.

Q2: What is the Lost Update Problem? Give an example with a schedule table.

The Lost Update Problem occurs when two transactions both read an item, then both update it. The second write overwrites the first, making T1's update disappear.

Step	T1 (Withdraw $200)	T2 (Deposit $300)
1	R(balance = 1000)	
2		R(balance = 1000)
3	balance := 1000 - 200 = 800	
4		balance := 1000 + 300 = 1300
5	W(balance = 800)	
6		W(balance = 1300) ← T1 LOST!

T2 overwrites T1's write. Final balance = 1300 (should be 1100). T1's withdrawal is lost.

Q3: What is a Dirty Read (Temporary Update Problem)? How does it occur?

A dirty read occurs when transaction T2 reads a data item that was written by T1, but T1 has not yet committed. If T1 later rolls back, T2 has read a value that 'never existed'.

Step	T1	T2
1	R(X=500)	

2	X := X - 200; W(X=300)	
3		R(X=300) ← DIRTY READ
4		X := 300+100; W(X=400); Commit
5	ROLLBACK (X should → 500)	
6	X is restored to 500, but T2 committed using 300!	

Prevention: Use READ COMMITTED isolation level or higher.

Q4: Draw and explain the transaction state diagram. What operations cause state transitions?

The state diagram has 5 states:

- Active: Initial state. BEGIN_TRANSACTION moves here. READ/WRITE operations occur. Stays here until all operations done.
- Partially Committed: END_TRANSACTION moves here. All operations complete, but changes not yet on disk.
- Committed: COMMIT_TRANSACTION moves here. Changes are permanent. Cannot be undone.
- Failed: Abort signal (from Active or Partially Committed) moves here. Something went wrong.
- Terminated: Final state from either Committed or Failed. Transaction leaves the system.

Transitions: Begin → Active → (Read/Write loop) → Partially Committed → Committed → Terminated (success path). Abort from Active or Partially Committed → Failed → Terminated (failure path).

Q5: Given the following schedule, determine if it is conflict serializable using a precedence graph: S: R1(X), W2(X), R1(Y), W1(Y), R2(Y), W2(Z)

Step 1: Identify conflicts:

- R1(X) vs W2(X): T2 writes X after T1 reads X → Edge T1 → T2 (RW conflict)
- W2(X) vs W1(Y): No conflict (different items)
- R1(Y) vs W1(Y): Same transaction — not a conflict
- W1(Y) vs R2(Y): T2 reads Y after T1 writes Y → Edge T1 → T2 (WR conflict)
- W1(Y) vs W2(Z): No conflict (different items)

Step 2: Build Precedence Graph:

Nodes: T1, T2 | Edges: T1 → T2 (two edges, same direction)

Step 3: Check for cycles:

No cycle exists (only T1 → T2 direction). ✓ Schedule IS conflict serializable!

Equivalent serial order: T1 → T2

Q6: What is the difference between a Recoverable, Cascadeless, and Strict schedule? Give an example of each.

Recoverable: Ti may commit only after all Tj that Ti has read from have committed.

Example (Recoverable): Example (Recoverable):

- T1: W(X) | T2: R(X), W(X), Commit | T1: Commit ✓ T1 commits AFTER T2

Cascadeless: Ti may only READ values written by already-committed transactions.

Example (Cascadeless): Example (Cascadeless):

- T1: W(X), Commit | T2: R(X) ✓ T2 reads only after T1 commits

Strict: Ti may neither read NOR write X until the last transaction that wrote X has committed/aborted.

- Strictest form. Allows recovery by simply restoring the before-image of data items.

Hierarchy: Strict ⊂ Cascadeless ⊂ Recoverable ⊂ All Schedules

Q7: Calculate the number of possible serial schedules for 4 transactions. If a schedule S is given with 4 transactions, how do you find the equivalent serial order?

Number of serial schedules = n! = 4! = 4 × 3 × 2 × 1 = 24 possible serial schedules.

To find equivalent serial order from a non-serial schedule:

- Step 1: Build the precedence (serialization) graph for S.
- Step 2: If the graph has NO cycle, perform a topological sort.
- Step 3: The topological sort gives the equivalent serial order.
- Step 4: If there IS a cycle, the schedule is NOT conflict serializable.

Example: If edges are T1→T2, T1→T3, T3→T4, T2→T4 → Serial order: T1, T2, T3, T4 OR T1, T3, T2, T4 (both are valid topological sorts).

Q8: Explain the four SQL isolation levels. Which anomalies does each prevent?

SQL defines 4 isolation levels:

- READ UNCOMMITTED: Allows dirty reads, non-repeatable reads, phantoms. Allows reading uncommitted changes. Rarely used.

- READ COMMITTED: Prevents dirty reads. Still allows non-repeatable reads and phantoms. Default in many DBMSs (e.g., Oracle, SQL Server).
- REPEATABLE READ: Prevents dirty reads and non-repeatable reads. Phantoms still possible. Default in MySQL InnoDB.
- SERIALIZABLE: Prevents ALL anomalies (dirty reads, non-repeatable reads, phantoms). Equivalent to running transactions one-by-one. Highest correctness, lowest concurrency.

Memory trick: As you go from READ UNCOMMITTED → SERIALIZABLE, you prevent more anomalies but reduce concurrency.

Q9: What is View Serializability? When is a schedule view serializable but NOT conflict serializable?

A schedule S is view serializable if it is view-equivalent to some serial schedule. Two schedules are view-equivalent if for every item X:

- Same initial read: Same transaction reads the initial value of X from the database.
- Same read-from: If Ti reads X written by Tj in S, the same must hold in the equivalent serial schedule.
- Same final write: The same transaction performs the last write on X.

A schedule can be view serializable but NOT conflict serializable when it contains BLIND WRITES (writes to an item without first reading it).

Example of view serializable but NOT conflict serializable:

T1	T2	T3
W(X)		
	W(X)	
		W(X)

This is view serializable (T3 does the final write in all possible serial equivalents) but has WW conflicts that would make it NOT conflict serializable if order matters.

Q10: What are the operations tracked in the system log? Why is 'force-writing' the log important?

The system log records the following for each transaction:

- [start_transaction, T] — transaction begins

- [read_item, T, X] — transaction T reads item X
- [write_item, T, X, old_value, new_value] — T writes X with before and after values
- [commit, T] — transaction T successfully commits
- [abort, T] — transaction T is aborted/rolled back

Force-writing the log: The log buffer must be written to disk BEFORE a transaction reaches its commit point. This ensures that even if the system crashes immediately after a commit, the log on disk contains the commit record. Without force-writing, a committed transaction might lose its changes after a crash.

UNDO: Uses old_value to reverse uncommitted writes. REDO: Uses new_value to redo committed writes lost due to crash.

Quick Reference Summary

Term	One-Line Definition
Transaction	Logical unit of work — all or nothing execution.
ACID	Atomicity, Consistency, Isolation, Durability.
Schedule	Ordered sequence of operations from multiple transactions.
Serial Schedule	Transactions run one after another, no interleaving. Always correct.
Conflict	Two ops from different Txns on same item, at least one write.
Conflict Serializable	Schedule equivalent to some serial schedule via conflict equivalence.
Precedence Graph	Graph to test serializability; no cycle → serializable.
View Serializable	Schedule with same initial reads, read-from, and final writes as a serial schedule.
Recoverable Schedule	Ti commits only after all Tj it read from have committed.
Cascadeless Schedule	Ti reads only from committed transactions (no dirty reads).
Strict Schedule	No read or write on X until last writer of X commits/aborts.
Lost Update	T2's write overwrites T1's update; T1's change vanishes.
Dirty Read	Transaction reads uncommitted data that is later rolled back.
Unrepeatable Read	Same item read twice gives different values.
Phantom Read	Re-running a query finds new rows inserted by another Txn.
System Log	Append-only file recording all transaction operations for recovery.
Commit Point	All ops done + log written → transaction can commit.
Snapshot Isolation	Transaction sees committed DB snapshot from when it started.

End of Chapter 20 Notes | Good Luck on Your Exam!